Identify bug

Step 1: Identifying the Bug

Problematic Code Snippet (merge function):

```
while (i < left.length) {
  result.push(left[i]);
  j++; // Error: j is increased instead of i
}
```

Observation:

- After merging matching elements from both arrays, any leftover elements from the **left** array still need to be added to the result.
- However, the loop increases **j** instead of **i**, which leads to incorrect behaviour. This mistake may cause the loop to run indefinitely or add incorrect values to the result.

Step 2: Using Verification Strategies

1. Collaborative Review

- I consulted classmates and used AI tools to check the merge function logic.
- The shared conclusion was that the remaining elements from the **left** array were not being added correctly because of the incorrect index update (**j++**).
- **Action Taken:** Replace **j++** with **i++** to fix the issue.

---

2. Testing Alternative Methods

- I implemented a typical iterative merge function and tested it with the same input arrays.
- The results showed that changing the increment to **i++**, as suggested by AI, generated the correct sorted output.
- **Lesson Learned:** Minor index errors can significantly affect recursive algorithms such as merge sort.

---

3. Careful Code Inspection

- I manually stepped through the algorithm using small arrays like **[3, 1, 4]**.
- During this process, I noticed that the final element from the left array was not added properly.

- This manual check confirmed that the correction suggested by AI was valid.

Step 3: Implementing the Corrected Solution

The corrected merge sort function updates the index properly when adding remaining elements from the left array.

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr;

  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));

  return merge(left, right);
}

function merge(left, right) {
  let result = [];
  let i = 0;
  let j = 0;

  while (i < left.length && j < right.length) {
    if (left[i] < right[j]) {
      result.push(left[i]);
      i++;
    } else {
      result.push(right[j]);
      j++;
    }
  }

  // Corrected remaining loops
  while (i < left.length) {
    result.push(left[i]);
    i++; // Correct increment
  }

  while (j < right.length) {
    result.push(right[j]);
    j++;
  }
```

```
  return result;
}

// Example test
console.log(mergeSort([3, 1, 4, 1, 5, 9, 2]));
// Output: [1, 1, 2, 3, 4, 5, 9]
```

Step 4: Reflection and Learning

1. Confidence in the Solution

After reviewing the fix with peers, checking AI suggestions, and manually tracing the code, I am confident that the merge sort implementation now works correctly.

---

2. Evaluating the AI Suggestion

The AI accurately detected the index mistake (**j++ instead of i++**). However, careful checking was still necessary, especially with loop behaviour, since small indexing errors can cause major issues in recursive functions.

---

3. Most Helpful Verification Method

Manual tracing proved to be the most effective strategy because it clearly revealed where the incorrect index update occurred. Peer and AI feedback further confirmed the correctness of the fix.

---

4. Key Lessons Learned

- Recursive algorithms depend heavily on correct index handling.
- Testing with small datasets makes it easier to identify logical errors.
- Combining AI assistance with manual validation improves both speed and accuracy when solving coding problems.